

Test cases: POST /api/xmod/calculate

X-Mod Lab: agent-generated, human-approved test cases. Practice project built from the California Workers' Compensation Experience Rating Plan effective Sept 1, 2025. **Not an official WCIRB tool.**

Plan: specs/plan-calculate-api.md · Tests: tests/agent/api/ · Branch agent/2026-10-04-calculate-api · Test run Oct 4, 2026, 12:54 PM

51

scenarios (all approved)

42

pass (6 partly implemented)

6

fail: expected, candidate defects

3

blocked on open questions

How expected values were set	Every expected value comes from a Plan rule (PLAN), arithmetic written out in the case (HAND-CALC), the API error contract (CONTRACT) or a documented assumption (ASSUMPTION). None was copied from what the app displayed.
PASS	The app behaves as the case expects.
FAIL	The app contradicts the expected value: a candidate defect (CD-1..CD-6). These tests are left failing on purpose as evidence.
BLOCKED	Not implemented: the expected value depends on an open question (Q1–Q20) the project owner has not answered yet.

Index

ID	Title	Family	P	Result
AG-NEG-1	Top-level JSON that is not an object (null, number, string)	API CONTRACT	2	PASS
AG-NEG-2	Empty, array and empty-object bodies with a JSON content type	INVALID INPUT	2	PASS
AG-NEG-3	Missing or wrong content type	API CONTRACT	2	PASS
AG-NEG-4	payroll container has the wrong type	INVALID INPUT	2	PASS
AG-NEG-5	Array entries that are not objects	INVALID INPUT	2	PASS
AG-NEG-6	Wrong HTTP methods and path variants on the calculate route	API CONTRACT	3	PASS
AG-NEG-7	Payroll values of the wrong type or format	INVALID INPUT	1	PASS
AG-NEG-8	Payroll that sums to zero, including negative zero	INVALID INPUT	2	PASS
AG-NEG-9	Loss amounts of the wrong type or missing	INVALID INPUT	1	PASS
AG-NEG-10	Class code variants (leading zeros, length, type, whitespace)	INVALID INPUT	1	PASS
AG-NEG-11	Class codes that are Object.prototype property names	INVALID INPUT	1	FAIL
AG-NEG-12	Claim id variants through the API	INVALID INPUT	3	PASS
AG-NEG-13	Unknown treatment value	INVALID INPUT	1	FAIL
AG-NEG-14	netIncurred on a claim whose treatment is none or absent	INVALID INPUT	2	PASS
AG-NEG-15	Boolean flags sent as strings or numbers	INVALID INPUT	1	FAIL
AG-NEG-16	contractMedical shapes	INVALID INPUT	2	PASS
AG-NEG-17	multiPerson and accidentId combinations	INTERACTION RULES	2	FAIL
AG-BND-1	Table II edge reached through payroll with fractional E (7,248 / 7,249)	BOUNDARY	1	PASS
AG-BND-2	Table II edge 27,391 / 27,392 through calculate with fractional E	BOUNDARY	2	PASS
AG-BND-3	\$250 floor at 0, 249, 250, 250.005, 251	BOUNDARY	1	PASS
AG-BND-4	Maximum loss value 174,999 / 175,000 / 175,001, also split across indemnity and medical	BOUNDARY	1	PASS
AG-BND-5	AL around the primary threshold (PT-1, PT, PT+1)	BOUNDARY	2	PASS
AG-BND-6	COVID window dates (inclusive 2019-12-01 to 2024-08-31)	BOUNDARY	1	PASS
AG-BND-7	netIncurred bounds on a subrogation claim	BOUNDARY	1	PASS

ID	Title	Family	P	Result
AG-BND-8	Multi-person accident Actual Primary cap edge (16,500 vs 16,501)	BOUNDARY	2	PASS
AG-BND-9	Request body size limit 262,144 vs 262,145 bytes	BOUNDARY	3	PASS
AG-EDGE-1	Floating-point sums stay exact in dollars	BOUNDARY	2	PASS
AG-EDGE-2	Sub-cent loss amounts	INVALID INPUT	3	PASS
AG-EDGE-3	Extreme magnitudes	INVALID INPUT	2	PASS
AG-EDGE-4	Unit confusion, dollars entered for a per-capita class	UNIT CONFUSION	1	PASS
AG-EDGE-5	Unit confusion, a head count entered for a dollar class, and fractional heads	UNIT CONFUSION	2	PASS
AG-EDGE-6	Duplicate payroll class lines are added, not rejected	INVALID INPUT	3	PASS
AG-EDGE-7	Claim ids differing only by case or trailing space	INVALID INPUT	3	BLOCKED
AG-EDGE-8	Non-compensable combined with a treatment	INTERACTION RULES	2	PASS
AG-EDGE-9	Rule precedence when death, EL+WC, treatment and joint overlap	INTERACTION RULES	3	PASS
AG-EDGE-10	multiPerson group of one, and COVID fields without their pair	INTERACTION RULES	3	PASS
AG-EDGE-11	accidentDate is not validated as a date	INVALID INPUT	1	FAIL
AG-SEC-1	Prototype-pollution keys in the body leave no trace	SECURITY AND ROBUSTNESS	1	PASS
AG-SEC-2	Markup, RTL, emoji and null bytes in text fields round-trip as inert data	SECURITY AND ROBUSTNESS	2	PASS
AG-SEC-3	Error messages echo user input without a length bound	SECURITY AND ROBUSTNESS	2	FAIL
AG-SEC-4	Security headers and JSON content type on error responses	SECURITY AND ROBUSTNESS	2	PASS
AG-SEC-5	No stack trace, file path or result fields on any failure	API CONTRACT	1	PASS
AG-SEC-6	X-Forwarded-For cannot be used to evade the rate limit	SECURITY AND ROBUSTNESS	3	PASS
AG-FLOW-1	Five identical requests in parallel	STATE AND FLOW	3	PASS
AG-FLOW-2	A failed request does not poison the next one	STATE AND FLOW	2	PASS
AG-FLOW-3	Outcome counters classify each failure	API CONTRACT	3	PASS
AG-FLOW-4	Parse failures do not consume rate-limit budget	API CONTRACT	2	BLOCKED
AG-FLOW-5	One shared rate-limit counter across three POST endpoints	API CONTRACT	2	BLOCKED
AG-FLOW-6	429 contract on the isolated app, error shape and no result	API CONTRACT	2	PASS
AG-SMOKE-1	Response schema of a successful calculation	API CONTRACT	2	PASS
AG-SMOKE-2	Optional collections omitted behave as empty	INVALID INPUT	3	PASS

A. NEGATIVE: request shape and content type

AG-NEG-1 Top-level JSON that is not an object (null, number, string) PASS

Layer: API · Family: API CONTRACT · Priority 2 · `tests/agent/api/negative-shape.spec.ts`

Preconditions	server up.
Steps	1. POST with <code>content-type: application/json</code> and raw body <code>null</code> . 2. Repeat with raw body <code>5</code> . 3. Repeat with raw body <code>"x"</code> .
Expected	each response is 400 with content-type <code>application/json</code> and body exactly <code>{"error":{"code":"MALFORMED_JSON","message":"<text>}}</code> . No <code>mod</code> key anywhere.
Expected source	CONTRACT (400 MALFORMED_JSON for unparsable body, docs/SECURITY_AND_A11Y.md section 1); the strict body parser rejects non-object roots. ASSUMPTION that MALFORMED_JSON (not MALFORMED_REQUEST) is the wanted code for valid-JSON-but-not-an-object, see Open question Q1.
Existing coverage	API-X <code>malformed JSON -> 400 MALFORMED_JSON</code> covers only <code>{not json}</code> .

AG-NEG-2 Empty, array and empty-object bodies with a JSON content type PASS

Layer: API · Family: INVALID INPUT · Priority 2 · `tests/agent/api/negative-shape.spec.ts`

Preconditions	none.
Steps	1. POST content-type <code>application/json</code> with no body at all. 2. POST <code>[]</code> . 3. POST <code>{}</code> .
Expected	each is 422 <code>NO_PAYROLL</code> ("At least one payroll line is required"), error shape only, no result fields.
Expected source	CONTRACT (US-06; <code>NO_PAYROLL</code> is the documented code for no payroll lines, API-X case "no payroll").
Existing coverage	API-X <code>US-06 no payroll -> 422 NO_PAYROLL</code> covers <code>{payroll:[],claims:{}}</code> only.

AG-NEG-3 Missing or wrong content type PASS

Layer: API · Family: API CONTRACT · Priority 2 · `tests/agent/api/negative-shape.spec.ts`

Preconditions	none.
Steps	1. POST valid REF JSON text with header <code>content-type: text/plain</code> . 2. POST <code>payroll=1</code> with <code>application/x-www-form-urlencoded</code> . 3. POST valid REF JSON text with no content-type header. 4. POST valid REF JSON text with <code>application/json; charset=utf-16</code> .
Expected	steps 1-3 return a 4xx (never 5xx, never 200) JSON error with a non-empty <code>error.code</code> and no result. Step 4 returns 400 MALFORMED_JSON or 415, JSON error shape.
Expected source	CONTRACT (error shape on every failure; existing test already pins "4xx, not 500"). The exact status/code is OPEN-QUESTION Q2.
Existing coverage	API-S <code>non-JSON content type is not parsed and is rejected without a 500</code> covers step 1 only.

AG-NEG-4 payroll container has the wrong type PASS

Layer: API · Family: INVALID INPUT · Priority 2 · tests/agent/api/negative-shape.spec.ts

Preconditions	none.
Steps	1. POST {"payroll":"abc","claims":[]} . 2. POST {"payroll":{},"claims":[]} . 3. POST {"payroll":5,"claims":[]} . 4. POST {"claims":[]} (payroll absent).
Expected	every response is a 4xx JSON error whose message does not contain the text "undefined". Steps 2-4 are 422 NO_PAYR OLL (no payroll lines). Step 1 is a 4xx shape error.
Expected source	CONTRACT (error shape, specific message, US-06 "invalid input rejected with a specific message"). Exact code for step 1 is OPEN-QUESTION Q3.
Existing coverage	none.

Partly implemented: step 1 blocked (Q3)

AG-NEG-5 Array entries that are not objects PASS

Layer: API · Family: INVALID INPUT · Priority 2 · tests/agent/api/negative-shape.spec.ts

Preconditions	none.
Steps	1. POST {"payroll":[null],"claims":[]} . 2. POST REF with "claims":[null] . 3. POST REF with "claims":[5], then [[]], then ["x"] . 4. POST REF with "claims":{} (object instead of array). 5. POST REF with "claims":"abc" .
Expected	no 5xx. null entries and the object claims (steps 1, 2, 4) are 400 MALFORMED_REQUEST ; entries of number, array and string type (step 3) and the string claims (step 5) are 422 BAD_CLAIM_ID . No result field.
Expected source	CONTRACT (400 MALFORMED_REQUEST for a body that is not a valid rating input; 422 BAD_CLAIM_ID for a claim without text id, US-06 AC "Claim ids must be non-empty text"). The 400-versus-422 split is ASSUMPTION (follows the app's TypeError mapping); see Q3.
Existing coverage	U-X rejects claim id %j (DEF-006) covers id types, not non-object claim entries; API-X payroll as a string does not 500 is the only shape test.

AG-NEG-6 Wrong HTTP methods and path variants on the calculate route PASS

Layer: API · Family: API CONTRACT · Priority 3 · tests/agent/api/negative-shape.spec.ts

Preconditions	none.
Steps	1. Send PUT, PATCH and DELETE to /api/xmod/calculate with a REF body. 2. Send OPTIONS and HEAD to the same path. 3. POST to /api/xmod/calculate/ (trailing slash), /API/XMOD/CALCULATE and /api/xmod/calculate?x=1 .
Expected	steps 1-2 return 404 with the JSON NOT_FOUND shape (HEAD has no body by HTTP rules) and no Allow header requirement; no 5xx. Step 3: same result as the plain route (200 with REF values) OR a JSON 404; never 5xx.
Expected source	CONTRACT (wrong method on a real route is JSON 404 NOT_FOUND, API-S). Step 3 is OPEN-QUESTION Q4.
Existing coverage	API-S wrong method on a real route -> JSON 404 NOT_FOUND covers GET only.

B. NEGATIVE: numeric and string field types

AG-NEG-7 Payroll values of the wrong type or format PASS

Layer: API · Family: INVALID INPUT · Priority 1 · tests/agent/api/negative-fields.spec.ts

Preconditions	none.
Steps	each a separate POST with class 0005 1. payroll "1000000" (numeric string). 2. payroll "1,000,000". 3. payroll "\$1000000". 4. payroll "1e6". 5. payroll "Infinity" and "NaN" as strings. 6. payroll null, true, [1000000], {}. 7. raw JSON text with payroll 1e999 (parses to Infinity). 8. raw JSON text with payroll NaN. 9. payroll -0.01.
Expected	steps 1-7 and 9: 422 BAD_PAYROLL ("Invalid payroll for 0005"). Step 8: 400 MALFORMED_JSON. No result field, no 500.
Expected source	CONTRACT (US-06 "Negative payroll or loss -> rejected"; code BAD_PAYROLL; only finite non-negative JSON numbers are payroll). Strings with separators/symbols are rejected because the API takes JSON numbers, not formatted text (ASSUMPTION, matches the UI which strips separators client-side, see Q5).
Existing coverage	API-X negative payroll, payroll as a string does not 500 (asserts only < 500). Steps 2-9 none.

AG-NEG-8 Payroll that sums to zero, including negative zero PASS

Layer: API · Family: INVALID INPUT · Priority 2 · tests/agent/api/negative-fields.spec.ts

Preconditions	none.
Steps	1. POST {"payroll":[{"classCode":"0005","payroll":-0}], "claims":[]}] as raw JSON text. 2. POST two lines 0005 payroll 0 and 3634 payroll 0.
Expected	both are 422 ZERO_EXPECTED ("Expected losses are zero; risk cannot be rated").
Expected source	CONTRACT (ZERO_EXPECTED documented in API-X; $E = 0 \times \text{ELR} / 100 = 0$ by HAND-CALC).
Existing coverage	API-X zero expected covers a single line of 0.

AG-NEG-9 Loss amounts of the wrong type or missing PASS

Layer: API · Family: INVALID INPUT · Priority 1 · tests/agent/api/negative-fields.spec.ts

Preconditions	REF risk with one claim {"id":"a","indemnity":X,"medical":0}.
Steps	X = "100", X absent, X null, X true, X [5], X -0.01; also medical = null with indemnity 100; also raw JSON 1e999 for indemnity.
Expected	each is 422 BAD_LOSS ("Invalid loss amount on claim a"). No result field.
Expected source	CONTRACT (US-06 "Negative payroll or loss -> rejected", code BAD_LOSS in API-X).
Existing coverage	API-X negative loss (indemnity -1 only); U-X rejects negative losses.

AG-NEG-10 Class code variants (leading zeros, length, type, whitespace) PASS

Layer: API · Family: INVALID INPUT · Priority 1 · tests/agent/api/negative-fields.spec.ts

Preconditions	none.
Steps	payroll 1,000,000, one POST each classCode "005", "00005", " 0005", "0005 ", "5", number 5, number 5.0, "", null, absent, "abcd".
Expected	each is 422 UNKNOWN_CLASS. No normalisation: "0005" is the only spelling of class 5. No 500.
Expected source	CONTRACT (US-06 "Unknown class code -> 422 UNKNOWN_CLASS"; data/table1_elr_dratios.json keys are 4-character strings, so only exact keys are known). Whether " 0005" should be trimmed is Q6.
Existing coverage	API-X unknown class (9999 only); API-X GET /api/classes/9999.

AG-NEG-11 Class codes that are Object.prototype property names FAIL

Layer: API · Family: INVALID INPUT · Priority 1 · tests/agent/api/negative-fields.spec.ts

Preconditions	none.
Steps	1. POST payroll classCode "__proto__", then "constructor", then "toString", then "hasOwnProperty" (payroll 1,000,000). 2. POST REF with contractMedical:[{"classCode":"constructor","incurred":1}]. 3. GET /api/classes/constructor and /api/classes/__proto__ (adjacent, same root cause).
Expected	steps 1-2: 422 UNKNOWN_CLASS with message naming the code. Step 3: 404 UNKNOWN_CLASS. Never 500, never 200.
Expected source	CONTRACT (US-06 "Unknown class code -> 422 UNKNOWN_CLASS"; error contract says 500 INTERNAL is only for unexpected server faults, and user input must not cause one).
Existing coverage	none.

Fails on purpose, candidate defect CD-1: class code "__proto__" (also "constructor", "toString") returns 500 INTERNAL instead of 422 UNKNOWN_CLASS.

AG-NEG-12 Claim id variants through the API PASS

Layer: API · Family: INVALID INPUT · Priority 3 · tests/agent/api/negative-fields.spec.ts

Preconditions	REF risk.
Steps	claim id "", " ", a 101-character string, absent, null, 7, ["a"], true.
Expected	each is 422 BAD_CLAIM_ID.
Expected source	CONTRACT (US-06 AC "Claim ids must be non-empty text of at most 100 characters"; DEF-006).
Existing coverage	U-X rejects claim id %j (DEF-006) (6 cases at unit level); API-S a non-string claim id is rejected (object only). API gap is small.

AG-NEG-13 Unknown treatment value **FAIL**

Layer: API · Family: INVALID INPUT · Priority 1 · tests/agent/api/negative-fields.spec.ts

Preconditions	REF risk, one claim indemnity 1,000, medical 0.
Steps	1. Add "treatment": "bogus" with no netIncurred. 2. Add "treatment": "bogus", "netIncurred": 500. 3. Add "treatment": "Subrogation" (wrong case) with netIncurred 500. 4. Add "treatment": null and "treatment": "".
Expected	steps 1-3 are 422 with a specific code naming the bad treatment (not BAD_NET, not 200). Step 4: null is treated as absent (200, AP 750) or rejected with 422; never silently takes another treatment's branch.
Expected source	CONTRACT (US-06 invalid input is rejected with a specific message; Treatment is the closed union "none subrogation fraud compromise joint" in src/engine/xmod.ts). New error code name is Q7.
Existing coverage	none.

Fails on purpose, candidate defect CD-4: an unknown treatment ("bogus") is not rejected as such: without a net it returns the misleading BAD_NET, with a net it is valued as an ordinary claim.

AG-NEG-14 netIncurred on a claim whose treatment is none or absent **PASS**

Layer: API · Family: INVALID INPUT · Priority 2 · tests/agent/api/negative-fields.spec.ts

Preconditions	REF risk, one claim indemnity 100, medical 0.
Steps	1. Add "netIncurred": 999 with no treatment. 2. Add "treatment": "none", "netIncurred": 999. 3. Add "netIncurred": -5 with no treatment.
Expected	either (a) 422 because net is only meaningful with a treatment, or (b) 200 with the net ignored: AL 100, AP 0 (100 <= 250 floor, Plan Sec VI R2 ordinary claim). Must not be 500 and must not change the result relative to the same claim without the net field.
Expected source	HAND-CALC for option (b) (AL 100 <= 250 so AP 0); the choice between (a) and (b) is OPEN-QUESTION Q8.
Existing coverage	none.

AG-NEG-15 Boolean flags sent as strings or numbers **FAIL**

Layer: API · Family: INVALID INPUT · Priority 1 · tests/agent/api/negative-fields.spec.ts

Preconditions	REF risk, one claim indemnity 20,000, medical 0, base case without flags: AL 20,000, AP = PT - 250 = 8,250, 25-point cap applies, mod 1.02.
Steps	separate POSTs, add one field to the claim 1. "nonCompensable": "false". 2. "death": "false". 3. "elAndWc": 0 and "elAndWc": "no". 4. "multiPerson": "false" with accidentId "A". 5. top-level "excludedUnauditedPayroll": "false" and "priorYearExperienceRated": "false". 6. "nonCompensable": null.
Expected	steps 1-5 are 422 with a specific type-error code (a string is not a boolean), OR at minimum the result equals the base case. Step 6: treated as absent, result equals the base case (AL 20,000, AP 8,250, mod 1.02). A string "false" must never change the rule applied.
Expected source	HAND-CALC for the base case (above) and CONTRACT (US-06: invalid input rejected, no silent wrong result). The code name is Q7.
Existing coverage	none.

Fails on purpose, candidate defect CD-2: flags are read by truthiness, so the string "false" switches a rule ON: nonCompensable:"false" excludes the claim (AP 0, mod 0.77 instead of AP 8,250, mod 1.02).

AG-NEG-16 contractMedical shapes **PASS**

Layer: API · Family: INVALID INPUT · Priority 2 · tests/agent/api/negative-fields.spec.ts

Preconditions	REF risk.
Steps	<code>contractMedical = "abc", {}, [null], [{"classCode":"0005"}]</code> (incurred absent), <code>[{"classCode":"0005","incurred":"5"}]</code> , <code>[{"classCode":"9999","incurred":1}]</code> , <code>[{"classCode":"0005","incurred":-1}]</code> , and <code>null</code> .
Expected	<code>null</code> is accepted and behaves as absent (REF result: E 20,200, mod 0.77). The seven others are 4xx JSON errors: unknown class <code>UNKNOWN_CLASS</code> , negative/missing/string incurred <code>BAD_LOSS</code> , wrong container or null entry 400 <code>MALFORMED_REQUEST</code> or 422 with a specific code. No 5xx.
Expected source	CONTRACT (US-06; <code>BAD_LOSS</code> and <code>UNKNOWN_CLASS</code> are the existing validation codes, <code>src/engine/xmod.ts validate</code>). The container-type code is Q3.
Existing coverage	none at API level; U-X has no contract-medical validation case.

AG-NEG-17 multiPerson and accidentId combinations **FAIL**

Layer: API · Family: INTERACTION RULES · Priority 2 · tests/agent/api/negative-fields.spec.ts

Preconditions	REF risk; claim a indemnity 20,000, medical 0.
Steps	<code>multiPerson:true</code> with no <code>accidentId</code>. <code>multiPerson:true,"accidentId":""</code>. <code>multiPerson:true,"accidentId":7</code> (number) and <code>["A"]</code>. <code>accidentId:"A1"</code> without <code>multiPerson</code> (single claim).
Expected	steps 1-2 are 422 <code>BAD_ACCIDENT</code> ("multiPerson requires accidentId"). Step 3 is a 422 (<code>accidentId</code> must be text, same reasoning as DEF-006). Step 4 is 200 and the claim is valued as an ordinary claim: one line with id <code>a</code> , AL 20,000, AP 8,250 (not renamed <code>accident:... </code>).
Expected source	CONTRACT for steps 1-2 (<code>BAD_ACCIDENT</code> in <code>src/engine/xmod.ts</code>); ASSUMPTION extending DEF-006 for step 3 (ids that are echoed must be bounded text); HAND-CALC for step 4 ($20,000 > PT\ 8,500, AP = 8,500 - 250 = 8,250$).
Existing coverage	none for <code>BAD_ACCIDENT</code> ; US-03 multi-person tests only cover valid groups (API-X <code>multi-person accident is one capped line</code>).

Fails on purpose, candidate defect CD-6: a numeric `accidentId` (7) is accepted and echoed (`accident:7`) instead of being rejected with 422.

C. BOUNDARY

AG-BND-1 Table II edge reached through payroll with fractional E (7,248 / 7,249) **PASS**

Layer: API · Family: BOUNDARY · Priority 1 · tests/agent/api/boundary.spec.ts

Preconditions	none.
Steps	- POST <code>[{"classCode":"0005","payroll":358836}]</code>, no claims. - POST the same with payroll 358837.
Expected	step 1: <code>expectedLosses</code> 7248.49, <code>primaryThreshold</code> 4500. Step 2: <code>expectedLosses</code> 7248.51, <code>primaryThreshold</code> 5000.
Expected source	HAND-CALC. $358,836 \times 2.02 = 717,672 + 7,176.72 = 724,848.72$; $/ 100 = 7,248.4872$, displays 7,248.49, rounds half-up to 7,248, band 0-7,248 -> 4,500. $358,837 \times 2.02 = 724,850.74$; $/ 100 = 7,248.5074$, displays 7,248.51, rounds to 7,249 -> 5,000. Rounding rule: ASSUMPTION RatingPolicy #2 (E to whole dollars before lookup); band edges from catalog E-05.
Existing coverage	catalog E-05 covers integers 7248/7249 through the lookup endpoint and the unit test; API-X has no calculate-level fractional case. E-09 covers 47,636.59 via lookup only.

AG-BND-2 Table II edge 27,391 / 27,392 through calculate with fractional E **PASS**Layer: API · Family: BOUNDARY · Priority 2 · `tests/agent/api/boundary.spec.ts`

Preconditions	none.
Steps	POST class 0005 with payroll 1,356,014, then 1,356,015, no claims.
Expected	payroll 1,356,014: expectedLosses 27391.48, PT 9500. Payroll 1,356,015: expectedLosses 27391.5, PT 10000.
Expected source	HAND-CALC. $1,356,014 \times 2.02 = 2,712,028 + 27,120.28 = 2,739,148.28$; $/ 100 = 27,391.4828$, rounds to 27,391 -> 9,500. $1,356,015 \times 2.02 = 2,739,150.30$; $/ 100 = 27,391.503$, rounds to 27,392 -> 10,000. ASSUMPTION RatingPolicy #2; edge from catalog E-07.
Existing coverage	catalog E-07 (integers via lookup); calculate-level none.

AG-BND-3 \$250 floor at 0, 249, 250, 250.005, 251 **PASS**Layer: API · Family: BOUNDARY · Priority 1 · `tests/agent/api/boundary.spec.ts`

Preconditions	REF risk (PT 8,500), one claim per POST, medical 0.
Steps	indemnity 0, 249, 250, 250.005, 251.
Expected	Actual Primary 0, 0, 0, 0.01, 1 respectively; Actual Losses equals the input (0, 249, 250, 250.005 shown as 250.01, 251). Claim with indemnity 0 is accepted (200).
Expected source	PLAN Sec VI R2 ordinary claim, floor: $AP = AL - 250$, never below 0. HAND-CALC: $250.005 - 250 = 0.005$, which rounds half-up to 0.01 at two decimals (ASSUMPTION: line amounts are rounded half-up to cents, <code>src/engine/xmod.ts out()</code> ; it is not in RatingPolicy, see Q9).
Existing coverage	API-X US-02 \$250 floor covers 250 and 251; U-X claims of \$250 or less count as zero-primary. Values 0, 249 and 250.005 are gaps.

AG-BND-4 Maximum loss value 174,999 / 175,000 / 175,001, also split across indemnity and medical **PASS**Layer: API · Family: BOUNDARY · Priority 1 · `tests/agent/api/boundary.spec.ts`

Preconditions	REF risk, one claim per POST.
Steps	(indemnity, medical) = (174999,0), (175000,0), (175001,0), (100000,75001), (0,175001).
Expected	Actual Losses 174,999 / 175,000 / 175,000 / 175,000 / 175,000. Actual Primary 8,250 in all five (every gross exceeds PT 8,500, so $AP = 8,500 - 250$).
Expected source	PLAN Sec VI R2, Actual Losses limited to the Maximum Loss Value 175,000 (<code>data/plan_constants.json</code>). HAND-CALC for AP as above.
Existing coverage	API-X US-02 ... MLV cap uses 250,000; U-X caps Actual Losses at the Maximum Loss Value, sums indemnity and medical before capping. The exact +-1 values at API level are a gap.

AG-BND-5 AL around the primary threshold (PT-1, PT, PT+1) **PASS**Layer: API · Family: BOUNDARY · Priority 2 · `tests/agent/api/boundary.spec.ts`

Preconditions	REF risk (PT 8,500), one claim per POST.
Steps	indemnity 8,499, 8,500, 8,501.
Expected	Actual Primary 8,249, 8,250, 8,250.
Expected source	PLAN Sec VI R2 ordinary claim: $AP = \min(AL, PT) - 250$. HAND-CALC: $8,499 - 250 = 8,249$; $8,500 - 250 = 8,250$; $\min(8,501, 8,500) - 250 = 8,250$.
Existing coverage	U-X property every claim's Ap is within $[0, PT - 250]$; the three exact values at API level none.

AG-BND-6 COVID window dates (inclusive 2019-12-01 to 2024-08-31) PASS

Layer: API · Family: BOUNDARY · Priority 1 · tests/agent/api/boundary.spec.ts

Preconditions	REF risk, one claim indemnity 50,000, medical 0, "catastropheNumber":12 . Included (not excluded) claim: AP 8,250, mod 1.02 (cap). Excluded claim: AP 0, mod = loss-free 0.77.
Steps	accidentDate = 2019-11-30, 2019-12-01, 2020-02-29, 2024-08-31, 2024-09-01.
Expected	2019-11-30 and 2024-09-01 are NOT excluded (AP 8,250, rule "VI.2 ordinary: AL above threshold"). 2019-12-01, 2020-02-29 and 2024-08-31 are excluded (AP 0, rule "VI.2.j COVID-19 (Cat. 12): excluded").
Expected source	PLAN Sec VI R2(j) as stated in web/stories.js US-03 ("COVID Cat.12 (12/1/2019-8/31/2024) claims are excluded"), both ends inclusive; HAND-CALC for AP and mod values above.
Existing coverage	API-X US-03 COVID Cat.12 claim is excluded (mid-window date only); U-X COVID exclusion needs Catastrophe No. 12 . Edge dates none.

AG-BND-7 netIncurred bounds on a subrogation claim PASS

Layer: API · Family: BOUNDARY · Priority 1 · tests/agent/api/boundary.spec.ts

Preconditions	REF risk, claim indemnity 10,000, medical 0, "treatment":"subrogation" .
Steps	netIncurred = 0, 9,999.99, 10,000, 10,000.01, -0.01; separately a claim with indemnity 0, medical 0 and netIncurred 0.
Expected	net 0: 200, AL 0, AP 0. net 10,000: 200, AL 10,000, AP 8,250. net 9,999.99: 200 (value not asserted until an oracle exists). net 10,000.01 and -0.01: 422 BAD_NET . Zero gross: 422 BAD_NET ("gross incurred is zero, ratio undefined").
Expected source	PLAN Sec VI R2(d): AP = max(0, min(gross, PT) x ratio - 250), ratio = net/gross. HAND-CALC: net 0: ratio 0, 0 - 250 -> floored 0; net = gross: ratio 1, min(10,000, 8,500) - 250 = 8,250. CONTRACT for BAD_NET (US-06 "Net incurred must not exceed gross incurred").
Existing coverage	API-X net exceeds gross (200 vs 100) and subrogation without net ; U-X subrogation/joint results floor at zero . Exact +-0.01 bounds and zero gross are gaps.

AG-BND-8 Multi-person accident Actual Primary cap edge (16,500 vs 16,501) PASS

Layer: API · Family: BOUNDARY · Priority 2 · tests/agent/api/boundary.spec.ts

Preconditions	REF risk (PT 8,500, cap 2 x PT - 500 = 16,500). All claims share "multiPerson":true,"accidentId":"A" , medical 0.
Steps	1. Claims a=20,000 and b=8,500. 2. Claims a=20,000, b=8,500 and c=251.
Expected	step 1: one line accident:A (a, b) , AP 16,500 (8,250 + 8,250, equal to the cap), AL 28,500. Step 2: one line accident:A (a, b, c) , AP 16,500 (uncapped sum 16,501 is capped), AL 28,751. Both: claimsWithPrimary 1, capApplied true, mod 1.02.
Expected source	PLAN Sec VI R2(a): multi-person AP limited to 2 x PT - 500 (US-03 AC "2xPT - \$500"). HAND-CALC: 8,250 + 8,250 + 1 = 16,501 > 16,500. Cap: raw mod (16,500 + 15,634.80)/20,200 = 1.5909 > 1.024 -> 1.02. The "grouped group counts once for the 25-point cap" behaviour is the documented fixed assumption in src/engine/xmod.ts RatingPolicy comment.
Existing coverage	U-X multi-person accident: Ap capped at 2*PT - 500 ; API-X multi-person accident is one capped line (three 20,000 claims). The exact equals/exceeds-by-one edge is a gap.

AG-BND-9 Request body size limit 262,144 vs 262,145 bytes **PASS**Layer: API · Family: BOUNDARY · Priority 3 · `tests/agent/api/boundary.spec.ts`

Preconditions	build a valid REF JSON body padded with an extra string field <code>pad</code> so the UTF-8 byte length is exactly 262,144, and another that is 262,145. (One request each; this is not load.)
Steps	POST each with content-type application/json.
Expected	262,144 bytes: 200 with REF values (extra field ignored). 262,145 bytes: 413 <code>PAYLOAD_TOO_LARGE</code> in the error shape.
Expected source	CONTRACT (body limit "256kb", 413 <code>PAYLOAD_TOO_LARGE</code> , docs/SECURITY_AND_A11Y.md). ASSUMPTION that "256kb" means 256 x 1024 bytes and that a body equal to the limit is accepted (body-parser compares length > limit).
Existing coverage	API-X and API-S <code>oversized body</code> (300,000 bytes only, far above the edge).

D. EDGE

AG-EDGE-1 Floating-point sums stay exact in dollars **PASS**Layer: API · Family: BOUNDARY · Priority 2 · `tests/agent/api/edge.spec.ts`

Preconditions	REF risk, one claim indemnity 100.1, medical 200.2.
Steps	POST and read the claim line.
Expected	actualLosses 300.3 exactly, actualPrimary 50.3 exactly, actualExcess 250. Not 300.29999999999995 or 50.29999999999995.
Expected source	HAND-CALC: $100.1 + 200.2 = 300.3$ in decimal arithmetic; $AP = 300.3 - 250 = 50.3$ (ordinary, within PT 8,500). Engine comment: "floats never touch a dollar amount". Mod check: $(50.3 + 15,634.80)/20,200 = 15,685.10/20,200 = 0.77654 \rightarrow 0.78$ (ASSUMPTION RatingPolicy #1, 2-decimal half-up).
Existing coverage	none.

AG-EDGE-2 Sub-cent loss amounts **PASS**Layer: API · Family: INVALID INPUT · Priority 3 · `tests/agent/api/edge.spec.ts`

Preconditions	REF risk.
Steps	one claim indemnity 0.005; another run with indemnity 250.004.
Expected	200 (or a documented 422 for more than 2 decimals). If 200: 0.005 -> actualLosses 0.01; 250.004 -> actualLosses 250, actualPrimary 0.
Expected source	ASSUMPTION (line amounts rounded half-up to 2 decimals, src/engine/xmod.ts <code>out()</code>); not one of the four RatingPolicy items); HAND-CALC: $250.004 - 250 = 0.004 \rightarrow 0.00$.
Existing coverage	none.

AG-EDGE-3 Extreme magnitudes **PASS**Layer: API · Family: INVALID INPUT · Priority 2 · `tests/agent/api/edge.spec.ts`

Preconditions	none.
Steps	raw JSON text 1. REF payroll 1e300, no claims. 2. REF payroll 1e-320, no claims. 3. REF with a claim indemnity 1.7e308, medical 1.7e308. 4. Same claim with <code>"treatment": "subrogation", "netIncurred": 1e308</code> (JS gross sums to Infinity).
Expected	never 5xx; any 200 response contains only finite JSON numbers. Step 3 (no treatment): 200 with AL 175,000 and AP 8,250 (capped at MLV). Steps 1-2 and 4: either a 200 with HAND-CALC numbers (step 1: $E = 1e300 \times 2.02 / 100 = 2.02e298$, top band PT 75,000) or a 422 with a specific code if a payroll ceiling is introduced.
Expected source	HAND-CALC (MLV cap, band table "and over"); CONTRACT (no 5xx). Whether a ceiling exists is OPEN-QUESTION Q10.
Existing coverage	none.

AG-EDGE-4 Unit confusion, dollars entered for a per-capita class **PASS**Layer: UI+API · Family: UNIT CONFUSION · Priority 1 · `tests/agent/api/edge.spec.ts`

Preconditions	none.
Steps	1. POST class 7707 payroll 100 (correct unit: 100 persons). 2. POST class 7707 payroll 1,000,000 (dollars typed by mistake). 3. In the UI, enter 1,000,000 for class 7707 and calculate; look for any hint that the figure is a head count.
Expected	step 1: expectedLosses 12,650, PT 6,500, perUnitBasis true (catalog E-02). Step 2: the API cannot tell the units apart; it returns expectedLosses 126,500,000.00 and PT 75,000 (top band "and over", E-08). Step 3: the UI should warn or label the unit for per-capita classes (OPEN-QUESTION Q11).
Expected source	HAND-CALC. 7707 ELR 126.50 per person, not divided by 100: $100 \times 126.50 = 12,650$; $1,000,000 \times 126.50 = 126,500,000 \geq 3,293,540 \rightarrow$ top band 75,000. Catalog E-02 and E-08.
Existing coverage	E-02 (correct unit only); no wrong-unit scenario.

Partly implemented: step 3 blocked (Q11)

AG-EDGE-5 Unit confusion, a head count entered for a dollar class, and fractional heads **PASS**Layer: API · Family: UNIT CONFUSION · Priority 2 · `tests/agent/api/edge.spec.ts`

Preconditions	none.
Steps	1. POST class 0005 payroll 100 (a head count typed into a dollar class). 2. POST class 7707 payroll 100.5 (half a person).
Expected	step 1: 200, expectedLosses 2.02, PT 4,500, eligible false, eligibilityReason mentions "< 10800" and "not rated prior year". Step 2: 200 with expectedLosses 12,713.25 (documented as accepted) or 422 if whole heads are required.
Expected source	HAND-CALC: $100 \times 2.02 / 100 = 2.02$ (first band, PT 4,500); $100.5 \times 126.50 = 12,713.25$. US-05 AC (eligible only when $E \geq 10,800$). Fractional heads: Q11.
Existing coverage	U-X per-capita classes are not divided by 100 (integer heads only).

AG-EDGE-6 Duplicate payroll class lines are added, not rejected **PASS**Layer: API · Family: INVALID INPUT · Priority 3 · `tests/agent/api/edge.spec.ts`

Preconditions	none.
Steps	POST two lines class 0005, payroll 500,000 each.
Expected	200, expectedLosses 20,200 (same as REF), <code>classes</code> has two entries.
Expected source	HAND-CALC: $2 \times (500,000 \times 2.02 / 100) = 2 \times 10,100 = 20,200$; catalog E-03 (several classes add up). Whether repeated class codes should instead be rejected is Q12.
Existing coverage	U-X <code>sums multiple classes</code> (different classes only).

AG-EDGE-7 Claim ids differing only by case or trailing space **BLOCKED**

Layer: API · Family: INVALID INPUT · Priority 3

Preconditions	REF risk.
Steps	POST two claims with ids <code>a / A</code> ; then <code>a / "a "</code> .
Expected	both are accepted as two distinct claims OR rejected <code>DUPLICATE_CLAIM</code> ; the decision is documented.
Expected source	OPEN-QUESTION Q13 (US-06 says "Duplicate claim ids rejected" without defining equality).
Existing coverage	API-X <code>duplicate claim ids</code> (exact match only).

Not implemented: waiting for answer to Q13

AG-EDGE-8 Non-compensable combined with a treatment **PASS**Layer: API · Family: INTERACTION RULES · Priority 2 · `tests/agent/api/edge.spec.ts`

Preconditions	REF risk, claim indemnity 20,000, medical 0, <code>"nonCompensable":true</code> .
Steps	- Add <code>"treatment":"subrogation"</code> with no <code>netIncurred</code>. - Add <code>"treatment":"subrogation", "netIncurred":30000</code> (above gross). - Add <code>"treatment":"subrogation", "netIncurred":5000</code>.
Expected	step 3: 200, claim excluded (AL 0, AP 0, rule "VI.2.c non-compensable: excluded", mod 0.77). Steps 1-2: either 422 <code>BA D_NET</code> (invalid data rejected even though the claim would be excluded) or 200 with the claim excluded; the rule must be documented.
Expected source	PLAN Sec VI R2(c) for step 3 (non-compensable contributes nothing; HAND-CALC mod = loss-free 0.77). Steps 1-2 are OPEN-QUESTION Q14.
Existing coverage	U-X <code>non-compensable claims contribute nothing</code> ; no combination case.

Partly implemented: steps 1-2 blocked (Q14)

AG-EDGE-9 Rule precedence when death, EL+WC, treatment and joint overlap **PASS**Layer: API · Family: INTERACTION RULES · Priority 3 · `tests/agent/api/edge.spec.ts`

Preconditions	REF risk (PT 8,500).
Steps	- Claim indemnity 1,000, <code>death:true, elAndWc:true</code>. - Claim indemnity 10,000, <code>elAndWc:true, treatment:"subrogation", netIncurred:5000</code>. - Claim indemnity 1,000, <code>death:true, treatment:"compromise", netIncurred:0</code>.
Expected	no 5xx; each result names exactly one rule. Step 3: HAND-CALC ratio 0 so AL 0 and AP 0 (rule VI.2.g). Steps 1-2 depend on precedence (observed: step 1 as VI.2.f death, AL 175,000, AP 8,250; step 2 as VI.2.d, AL 5,000, AP 4,000).
Expected source	HAND-CALC for step 3 (ratio 0). Steps 1-2: OPEN-QUESTION Q14 (Plan precedence not in repo).
Existing coverage	U-X has each rule alone (<code>EL + WC claim, death with compromise</code>).

Partly implemented: steps 1-2 blocked (Q14)

AG-EDGE-10 multiPerson group of one, and COVID fields without their pair PASS

Layer: API · Family: INTERACTION RULES · Priority 3 · tests/agent/api/edge.spec.ts

Preconditions	REF risk, claim indemnity 20,000 (base: AP 8,250, mod 1.02).
Steps	1. multiPerson:true,accidentId:"A1" on this single claim. 2. catastropheNumber:12 with no accidentDate. 3. accidentDate:"2021-06-01" with no catastropheNumber. 4. catastropheNumber:"12" (string) with accidentDate 2021-06-01.
Expected	step 1: one line named accident:A1 (a) , AP 8,250 (below the 16,500 cap), mod 1.02. Steps 2-3: not excluded (AP 8,250). Step 4: a string 12 must either be rejected 422 or treated the same as number 12 (excluded, AP 0); it must not silently remain included.
Expected source	PLAN Sec VI R2(j) (exclusion requires Catastrophe No. 12 AND an accident date in the window) for steps 2-3; HAND-CALC for AP. Step 4: ASSUMPTION (type strictness) Q7.
Existing coverage	U-X COVID exclusion needs Catastrophe No. 12 .

Partly implemented: step 4 blocked (Q7)

AG-EDGE-11 accidentDate is not validated as a date FAIL

Layer: API · Family: INVALID INPUT · Priority 1 · tests/agent/api/edge.spec.ts

Preconditions	REF risk, claim indemnity 50,000, catastropheNumber:12 .
Steps	accidentDate = "2024-08-31T12:00:00Z" , "2020-02-30" , "zzzz" , "2020-6-1" , 20200601 (number), "" .
Expected	422 for each value that is not a real ISO yyyy-mm-dd date. At minimum, "2024-08-31T12:00:00Z" (a moment on the last day of the window) must be classified like "2024-08-31" , that is excluded (AP 0, mod 0.77).
Expected source	CONTRACT (src/engine/xmod.ts documents accidentDate?: string // ISO yyyy-mm-dd ; US-06 invalid input rejected); PLAN window end 2024-08-31 inclusive (web/stories.js US-03). HAND-CALC for outcomes above.
Existing coverage	none (docs/ENGINE_PERIOD_RULES.md validates dates only on the experience-period endpoint).

Fails on purpose, candidate defect CD-3: accidentDate is compared as text: the impossible date "2020-02-30" is accepted (and treated as inside the COVID window) instead of 422.

E. SECURITY AND ROBUSTNESS

AG-SEC-1 Prototype-pollution keys in the body leave no trace PASS

Layer: API · Family: SECURITY AND ROBUSTNESS · Priority 1 · tests/agent/api/security.spec.ts

Preconditions	none.
Steps	1. POST raw JSON {"payroll":[{"classCode":"0005","payroll":1000000},"claims":[],"__proto__":{"polluted":1}} . 2. POST REF with "constructor":{"prototype":{"polluted":1}} . 3. POST a claim {"id":"a","indemnity":1,"medical":0,"__proto__":{"nonCompensable":true,"death":true}} as raw JSON text. 4. Then POST a clean claim {"id":"b","indemnity":20000,"medical":0} and a clean REF. 5. GET /api/health .
Expected	steps 1-3 are 200 with REF values (E 20,200, mod 0.77; step 3 claim AL 1, AP 0, rule "VI.2 ordinary: AL <= \$250", not excluded and not a death). Step 4: claim b is AL 20,000, AP 8,250 (not excluded, not death, so no earlier request polluted object.prototype); REF mod 0.77. Step 5: unchanged {status:"ok", version} .
Expected source	HAND-CALC (REF values; AL 1 <= 250 gives AP 0; 20,000 > 8,500 gives AP 8,250). CONTRACT (extra unknown fields are ignored).
Existing coverage	none.

AG-SEC-2 Markup, RTL, emoji and null bytes in text fields round-trip as inert data PASSLayer: API · Family: SECURITY AND ROBUSTNESS · Priority 2 · `tests/agent/api/security.spec.ts`

Preconditions	none.
Steps	1. Claim id <code>"\u202Eabc\u0000🐼"</code> (RTL override, NUL, emoji). 2. Claim id <code>""</code> . 3. Claim id of exactly 100 Unicode code units of "é" and one of 101. 4. accidentId <code>"<svg onload=alert(1)>"</code> on two grouped claims.
Expected	steps 1-2 and 4: 200, the same string comes back unchanged in <code>claims[].id</code> (step 4: <code>accident:<svg onload=alert(1)> (a, b)</code>), content-type application/json, x-content-type-options: nosniff. Step 3: 100 accepted, 101 is 422 <code>BAD_CLAIM_ID</code> (length is counted in UTF-16 code units, <code>String.length</code>).
Expected source	CONTRACT (API-S "injection-style strings are returned as data"; US-06 AC length 100). Code-unit counting is ASSUMPTION Q15.
Existing coverage	API-S injection-style strings are returned as data, never interpreted; ui <code>tests/ui/security.spec.ts</code> XSS (four payloads in claim ids). NUL, RTL, emoji, accidentId and length-by-code-unit are gaps.

AG-SEC-3 Error messages echo user input without a length bound FAILLayer: API · Family: SECURITY AND ROBUSTNESS · Priority 2 · `tests/agent/api/security.spec.ts`

Preconditions	none.
Steps	POST payroll classCode <code>"<script>" + "x" x 100000</code> (one request, 100 KB).
Expected	422 <code>UNKNOWN_CLASS</code> whose message is bounded (for example the first 20-50 characters, or no echo); the response is not larger than a few hundred bytes. No HTML rendering risk because content-type is JSON with nosniff.
Expected source	ASSUMPTION (same rationale as DEF-006: values echoed back must be bounded text). Bound is Q16.
Existing coverage	none.

Fails on purpose, candidate defect CD-6: a 100 KB class code is echoed back in full: the 422 body is 100,074 bytes instead of a short bounded message.

AG-SEC-4 Security headers and JSON content type on error responses PASSLayer: API · Family: SECURITY AND ROBUSTNESS · Priority 2 · `tests/agent/api/security.spec.ts`

Preconditions	none.
Steps	for each of 422 (unknown class), 400 (malformed JSON), 404 (unknown <code>/api/xmod/<script>alert(1)</script></code>), 413 (300 KB body), read headers.
Expected	every response has content-type application/json, x-content-type-options: nosniff, content-security-policy containing <code>default-src 'self'</code> and <code>frame-ancestors 'none'</code> , <code>referrer-policy: no-referrer</code> , <code>strict-transport-security</code> , and no <code>x-powered-by</code> . The 404 message for the markup path shows the URL percent-encoded (<code>%3Cscript%3E...</code>), not raw markup.
Expected source	CONTRACT (helmet applied before every route, docs/SECURITY_AND_A11Y.md section 1).
Existing coverage	API-S header test covers <code>/</code> and <code>/api/health</code> only; error responses unchecked.

AG-SEC-5 No stack trace, file path or result fields on any failure **PASS**

Layer: API · Family: API CONTRACT · Priority 1 · tests/agent/api/security.spec.ts

Preconditions	a list of every failing request from AG-NEG-1 to AG-NEG-17, AG-SEC-3 and AG-EDGE-11.
Steps	replay them once each (no more than 40 requests), parse each response.
Expected	every response parses as JSON with exactly one top-level key <code>error</code> holding exactly <code>code</code> (uppercase snake-case) and <code>message</code> (string). The text contains none of: <code>node_modules</code> , <code>at</code> +identifier, <code>C:\</code> , <code>/home/</code> , <code>.ts:</code> , <code><html</code> , <code>Type Error</code> , <code>stack</code> . No key <code>mod</code> , <code>result</code> , <code>claims</code> or <code>expectedLosses</code> appears. The 500 body for any case that still produces one is exactly <code>{"error":{"code":"INTERNAL","message":"Unexpected error"}}</code> .
Expected source	CONTRACT (docs/SECURITY_AND_A11Y.md: every error is <code>{error:{code,message}}</code> , no stack can reach a client; API-X "never return a result alongside an error").
Existing coverage	API-S <code>error contract</code> never leaks <code>internals</code> (5 cases); API-X validation table (8 cases). This scenario widens the net to every shape-error path.

AG-SEC-6 X-Forwarded-For cannot be used to evade the rate limit **PASS**

Layer: API · Family: SECURITY AND ROBUSTNESS · Priority 3 · tests/agent/api/security.spec.ts

Preconditions	isolated in-process app as in API-S, <code>createApp({rateLimit:{limit:3>windowMs:60000}})</code> , <code>TRUST_PROXY</code> unset (default 0). Never against the live server.
Steps	send 3 valid calculate requests with <code>X-Forwarded-For: 1.1.1.1, 2.2.2.2, 3.3.3.3</code> ; then a 4th with <code>X-Forwarded-For: 4.4.4.4</code> .
Expected	4th response is 429 <code>RATE_LIMITED</code> with <code>Retry-After: 60</code> ; the spoofed header did not create a new bucket.
Expected source	CONTRACT (limit is per client IP; <code>TRUST_PROXY</code> only when behind a proxy, docs/SECURITY_AND_A11Y.md section 1).
Existing coverage	API-S rate limit test (single client, no header).

F. STATE AND FLOW (API)

AG-FLOW-1 Five identical requests in parallel **PASS**

Layer: API · Family: STATE AND FLOW · Priority 3 · tests/agent/api/flow.spec.ts

Preconditions	note <code>/api/stats</code> ok count before.
Steps	fire 5 identical REF POSTs concurrently (a small, fixed number, not a load test); read <code>/api/stats</code> again.
Expected	five 200 responses with byte-identical bodies (mod 0.77, E 20,200, PT 8,500); <code>calculations.ok</code> increased by exactly 5.
Expected source	HAND-CALC (REF) and CONTRACT (<code>xmod_calculations_total</code> counts outcome ok per successful calculation, <code>src/server/app.ts</code>).
Existing coverage	API-X <code>US-08 /api/stats ... move after a calculation</code> (single request, <code>greaterThan</code>).

AG-FLOW-2 A failed request does not poison the next one **PASS**

Layer: API · Family: STATE AND FLOW · Priority 2 · tests/agent/api/flow.spec.ts

Preconditions	none.
Steps	POST the invalid body from AG-NEG-11 (<code>classCode constructor</code>), then malformed JSON, then REF; repeat REF twice.
Expected	the REF responses after the failures are identical to a REF response before them (mod 0.77, PT 8,500) and contain no <code>error</code> key.
Expected source	HAND-CALC (REF); CONTRACT (stateless endpoint).
Existing coverage	none.

AG-FLOW-3 Outcome counters classify each failure **PASS**

Layer: API · Family: API CONTRACT · Priority 3 · tests/agent/api/flow.spec.ts

Preconditions	read /api/stats calculations counters.
Steps	send one valid REF, one 422 (unknown class 9999), one 400 MALFORMED_REQUEST (claims:[null]), one malformed JSON body; read counters after each.
Expected	ok +1, validation_error +2 (the 422 and the 400 shape error); malformed JSON does not change the calculation counters. The request histogram does record all four (route="/api/xmod/calculate" , statuses 200, 422, 400, 400).
Expected source	ASSUMPTION (code in src/server/app.ts increments only inside the route handler; parse failures never reach it; DEF-004 requires them in the HTTP histogram). Whether parse failures should count in calculations is Q17.
Existing coverage	API-X body-parser errors are labelled with their endpoint ; U stats tests.

Partly implemented: malformed-JSON counter part blocked (Q17)

AG-FLOW-4 Parse failures do not consume rate-limit budget **BLOCKED**

Layer: API · Family: API CONTRACT · Priority 2

Preconditions	isolated in-process app with limit 3 (as API-S). Not on the live server.
Steps	send 5 malformed-JSON POSTs, then 3 valid POSTs, then a 4th valid POST.
Expected	as built the 5 malformed requests are never limited (limiter runs after the body parser) and the 4th valid request is 429. Expected (if the rule is "all POSTs to calculate"): the 4th, 5th... malformed requests are already 429.
Expected source	OPEN-QUESTION Q18 (docs say the limit covers "POST /api/xmod/calculate" without saying whether parse failures count). Observed live: the RateLimit headers are absent on 400 MALFORMED_JSON responses and the remaining counter did not move.
Existing coverage	API-S rate limit test (valid requests only).

Not implemented: waiting for answer to Q18

AG-FLOW-5 One shared rate-limit counter across three POST endpoints **BLOCKED**

Layer: API · Family: API CONTRACT · Priority 2

Preconditions	isolated in-process app with limit 3. Not on the live server.
Steps	POST calculate twice, POST /api/xmod/experience-period (any 422-producing body) once, then POST calculate again.
Expected	as built the 4th request is 429 because /api/xmod/experience-period and /api/xmod/rating-effective-date reuse the same limiter instance. Expected (if documented per endpoint): 200.
Expected source	OPEN-QUESTION Q18 (docs/SECURITY_AND_A11Y.md says only calculate is limited; the code shares one counter).
Existing coverage	none.

Not implemented: waiting for answer to Q18

AG-FLOW-6 429 contract on the isolated app, error shape and no result PASS

Layer: API · Family: API CONTRACT · Priority 2 · tests/agent/api/flow.spec.ts

Preconditions	isolated in-process app with limit 3.
Steps	exhaust the limit with 422 requests (invalid class) rather than valid ones; then send one valid REF.
Expected	4th request is 429 RATE_LIMITED with Retry-After: 60, content-type application/json, body exactly {"error":{"code":"RATE_LIMITED","message":"Too many calculation requests; limit is 3 per 60s"}}, no mod.Invalid requests consume budget (they count the same as valid ones).
Expected source	CONTRACT (429 RATE_LIMITED, docs/SECURITY_AND_A11Y.md). Message text copied from src/server/app.ts handler; the "invalid requests count" rule is ASSUMPTION (observed live: the remaining counter dropped on 400/422 responses that reach the route).
Existing coverage	API-S calculate returns 429 RATE_LIMITED after the limit (valid requests only, 429 body shape checked). Only the "failed requests count" part is new.

G. SMOKE (gaps only)

AG-SMOKE-1 Response schema of a successful calculation PASS

Layer: API · Family: API CONTRACT · Priority 2 · tests/agent/api/smoke.spec.ts

Preconditions	none.
Steps	POST REF; inspect keys and JSON types.
Expected	exactly the documented keys of RatingResult : planEffective "2025-09-01"; numbers for expectedLosses 20200, primaryThreshold 8500, expectedPrimary 4565.2, expectedExcess 15634.8, actualPrimary 0, actualLosses 0, actualExcess 0, maximumLossValue 175000, mod 0.77, lossFreeMod 0.77; strings for modUnrounded, modBeforeCap, lossFreeModUnrounded (all "0.774"); booleans capApplied false, eligible true; claimsWithPrimary 0; arrays classes (1 entry, perUnitBasis false, elr "2.02") and claims (empty); policy {modDecimals:2, roundExpectedForBand:true}. Content-type application/json; charset=utf-8; ratelimit and ratelimit-policy headers present.
Expected source	HAND-CALC (REF arithmetic at the top; Ep = 20,200 x D-ratio from Table I, value 4,565.20 pinned in API-X US-01, so not re-derived here); ASSUMPTION RatingPolicy #1/#2 for policy; CONTRACT for rate-limit headers.
Existing coverage	API-X US-01 loss-free reference risk checks 6 fields via toMatchObject; no key-set or type check.

AG-SMOKE-2 Optional collections omitted behave as empty PASS

Layer: API · Family: INVALID INPUT · Priority 3 · tests/agent/api/smoke.spec.ts

Preconditions	none.
Steps	POST {"payroll":[{"classCode":"constructor","payroll":1000000}]} (claims absent), then the same with "claims":null, then with "claims":[],"contractMedical":null.
Expected	each result equals the REF result (mod 0.77, E 20,200), or claims:null is rejected 4xx; never 5xx.
Expected source	HAND-CALC (REF). claims omitted is accepted because the engine defaults it (input.claims ?? []); claims:null accepted-or-rejected is Q19.
Existing coverage	none.

Candidate defects

Each has expected-versus-observed evidence from the live app (<http://localhost:3000>, 2026-10-03). I did not adjust any expectation to match.

- CD-1 (Major): class codes named like Object.prototype members crash or leak through validation.
- Steps: POST {"payroll":[{"classCode":"constructor","payroll":1000000}],"claims":[]}. Repeat with __proto__ and toString. POST REF with contractMedical:[{"classCode":"constructor","incurred":1}]. GET /api/classes/constructor.
- Observed: calculate returns 500 INTERNAL for the three payroll codes; contractMedical returns 400 MALFORMED_REQUEST; /api/classes/constructor returns 200.

- Expected: 422 UNKNOWN_CLASS (calculate) and 404 UNKNOWN_CLASS (GET).
- Source: CONTRACT (US-06 AC "Unknown class code -> 422 UNKNOWN_CLASS"; 500 reserved for server faults). Likely root cause: `T1.classes[cod` e] is an object lookup that finds inherited keys. Scenario AG-NEG-11.
- Note: the 500 body itself is clean (Unexpected error), so there is no leak; the defect is the wrong status and code, and that user input can trigger server_error metrics.
- CD-2 (Major, wrong mod): boolean flags are tested by truthiness, so the string "false" turns the rule on.
- Steps: REF with claim { "id": "a", "indemnity": 20000, "medical": 0, "nonCompensable": "false" }; then the same with "death": "false" .
- Observed: first gives AL 0, AP 0, rule "VI.2.c non-compensable: excluded", mod 0.77. Second gives AL 175,000 and rule "VI.2.f death".
- Expected: rejected 422, or at worst the ordinary result AL 20,000, AP 8,250, mod 1.02.
- Source: HAND-CALC (20,000 > PT 8,500 so AP = 8,250; cap value 1.024) and US-06. Scenario AG-NEG-15.
- CD-3 (Major, wrong mod at a window boundary): accidentDate is compared as a plain string.
- Steps: claim indemnity 50,000, catastropheNumber 12, accidentDate "2024-08-31T12:00:00Z" .
- Observed: not excluded (AP 8,250, mod 1.02) although "2024-08-31" is excluded (AP 0, mod 0.77). "2020-02-30" (impossible date) is excluded; "z zzz" and the number 20200601 are silently ignored.
- Expected: invalid dates rejected 422; a timestamp on 2024-08-31 handled the same as the date.
- Source: CONTRACT (type comment "ISO yyyy-mm-dd"), PLAN window inclusive of 8/31/2024 (web/stories.js US-03). Scenarios AG-EDGE-11, AG-BND-6.
- CD-4 (Minor): an unknown treatment string is accepted.
- Steps: claim indemnity 1,000 with "treatment": "bogus", "netIncurred": 500 .
- Observed: 200, valued as an ordinary claim (AP 750, mod 0.81 observed), net ignored. Without netIncurred the same input gives 422 BAD_NET , which names the wrong problem.
- Expected: 422 naming the invalid treatment.
- Source: CONTRACT (US-06; closed union in src/engine/xmod.ts). Scenario AG-NEG-13.
- CD-5 (Minor): payroll given as a string is iterated per character.
- Steps: POST { "payroll": "abc", "claims": [] } .
- Observed: 422 UNKNOWN_CLASS "Unknown class code undefined".
- Expected: a message that does not claim a class code undefined ; a shape error.
- Source: CONTRACT (specific message, US-06). Scenario AG-NEG-4.
- CD-6 (Minor): input echoed without bound or type check.
- Steps: (a) classCode "<script>" + "x" x 100000 ; (b) multiPerson claims with accidentId of 58,000 characters, and with accidentId number 7.
- Observed: (a) 422 body of 100,074 bytes; (b) 200, id echoed as accident:... ; number accepted and echoed as accident:7 (a) .
- Expected: bounded message; accidentId text of bounded length (as for claim ids, DEF-006).
- Source: ASSUMPTION (DEF-006 rationale). Scenarios AG-SEC-3, AG-NEG-17.
- Not re-raised: DEF-009 (contract medical counted for the 25-point cap) is already logged. No scenario in this plan asserts a contract-medical cap result.

Open questions for the human

- Q1: A body that is valid JSON but not an object (null , 5 , "x"): keep 400 MALFORMED_JSON, or return 422/400 MALFORMED_REQUEST? (AG-NEG-1)
- Q2: Wrong or missing content type: should this be 415 with its own code instead of 400 MALFORMED_REQUEST ("not a valid rating input")? (AG-NEG-3)
- Q3: What code should shape errors use (payroll/claims/contractMedical wrong container type): 400 MALFORMED_REQUEST everywhere, or 422 with specific codes? (AG-NEG-4, 5, 16)
- Q4: Should /api/xmod/calculate/ , upper-case path and query strings be accepted (Express defaults), and should a wrong method return 405 with Allow instead of 404? (AG-NEG-6)
- Q5: Is the API meant to accept only JSON numbers for money (strings like "1,000,000" rejected), leaving all formatting to the UI? (AG-NEG-7)
- Q6: Should class codes be trimmed or zero-padded ("5", " 0005") or must they match exactly? (AG-NEG-10)
- Q7: Names of new error codes for wrong type of flags, unknown treatment and bad accidentId/accidentDate (suggest BAD_TYPE, BAD_TREATMENT, BAD_ACCIDENT, BAD_DATE). (AG-NEG-13, 15, 17; AG-EDGE-10, 11)
- Q8: netIncurred supplied with treatment none/absent: reject, or ignore? (AG-NEG-14)
- Q9: Are amounts limited to 2 decimals, and is half-up cent rounding of claim lines (currently code behaviour, not in RatingPolicy) an accepted assumption? (AG-BND-3, AG-EDGE-2)
- Q10: Is there a ceiling on payroll or loss amounts (observed: 1e300 accepted, mod 0.19)? (AG-EDGE-3)
- Q11: For per-capita classes (7707, 7722, 8278, 8631) should the API or UI reject or warn on obviously dollar-sized values and fractional persons? (AG-EDGE-4, 5)
- Q12: Repeated class codes in payroll: sum (current) or reject? (AG-EDGE-6)
- Q13: Are claim ids compared case-sensitively and without trimming? (AG-EDGE-7)
- Q14: Plan precedence when a claim is non-compensable AND has a treatment, or is death AND EL+WC, or EL+WC AND subrogation. The Plan text is not in the repo; I cannot derive these. Also should invalid treatment data on an excluded claim still be rejected? (AG-EDGE-8, 9)
- Q15: Is the 100-character claim id limit counted in UTF-16 code units (current) or characters? (AG-SEC-2)

- Q16: What bound should error messages apply to echoed input? (AG-SEC-3)
- Q17: Should malformed-JSON requests count in `xmod_calculations_total{outcome=...}` ? (AG-FLOW-3)
- Q18: Should the rate limit cover parse failures, and should `/api/xmod/experience-period` and `/api/xmod/rating-effective-date` share the calculate counter (code does; docs list only calculate)? (AG-FLOW-4, 5)
- Q19: Is `"claims": null` (or `contractMedical: null`) valid input meaning "none"? (AG-SMOKE-2)
- Q20: There is no oracle (`tests/e2e/oracle.json` , `tools/oracle.py`). Do you want one built before the generator writes tests, so that mod values at half-up rounding boundaries and Table I D-ratio sums (for example scenarios needing Expected Primary for classes other than 0005) can use ORACLE instead of HAND-CALC?